

ToxicReels Production Scan and Lightsail Deployment Notes

Date: 2026-05-29

Repository: GodsplanElot/Tox

Production backend: <https://api.toxicreels.com>

Frontend: <https://toxicreels.com>

Current production branch baseline: main at 208486a

1. Executive Summary

The project is now deployed in a workable MVP production shape. The backend runs on AWS Lightsail through Docker Compose, Gunicorn, Nginx, HTTPS via Certbot, PostgreSQL in Docker, and S3 for uploaded media. The Vercel frontend is designed to talk to the backend through `VITE_API_BASE_URL=https://api.toxicreels.com/api`.

The application architecture is clear: Django REST Framework provides read-only public content APIs and authenticated user APIs; the Django admin acts as the internal CMS; React/Vite on Vercel is the public UI. Movies and series are status-controlled through draft, pending review, rejected, and published states. Admin roles are split between superusers, Editors, and Workers.

The system is close to production-ready for MVP use, but there are still hardening items to address before relying on it heavily: rotate exposed secrets, fix Google OAuth production client configuration, add CloudFront or a media delivery strategy, add a health endpoint, and tighten a few permission/data edge cases.

2. Current Build Shape Observed

Backend:

- Django 6.0.3
- Django REST Framework
- SimpleJWT authentication
- django-cors-headers
- django-storages + boto3 for S3
- Gunicorn inside Docker
- PostgreSQL 16 in Docker Compose
- Nginx reverse proxy on Lightsail
- Certbot-managed HTTPS for api.toxicreels.com
- S3 bucket for media: `toxicreels-prod-media`

Frontend:

- React 19 + Vite
- Hosted on Vercel
- API URL controlled by `VITE_API_BASE_URL`
- Media base controlled by `VITE_MEDIA_BASE_URL`
- Google Identity Services button using `VITE_GOOGLE_CLIENT_ID`
- JWT access/refresh tokens stored in `localStorage`

Admin/CMS:

- Django admin redesigned with custom admin dashboard styling
- Movie workflow through draft, pending review, published, rejected
- Series workflow includes Series, Season, Episode hierarchy
- Editors can review/publish content
- Workers can upload draft/pending content and have upload tracking fields

3. Verification Results From Local Scan

Passed:

- Full backend test suite: 36 tests OK
- Migration check: no changes detected
- Production Docker Compose config validates
- Frontend production build succeeds
- Django deploy check has only one warning: `SECURE_HSTS_PRELOAD` is false

The HSTS preload warning is acceptable for first production launch. Preload should only be enabled after the domain and subdomains are permanently stable on HTTPS.

4. Path From Local Project To Lightsail

Major steps completed:

1. Created AWS Lightsail Ubuntu instance.
2. Installed Docker and Docker Compose plugin.
3. Cloned the GitHub repository into /srv/toxicreels/Tox.
4. Added backend/Dockerfile for Django/Gunicorn.
5. Added docker-compose.prod.yml with web and postgres services only.
6. Created backend/.env on the server from the production template.
7. Configured PostgreSQL to use service host postgres and port 5432.
8. Configured S3 production variables for real AWS S3, not MinIO.
9. Started containers with docker compose --env-file backend/.env -f docker-compose.prod.yml up -d --build.
10. Ran migrations, collectstatic, and createsuperuser.
11. Configured Nginx for api.toxicreels.com.
12. Installed SSL with Certbot.
13. Added Nginx static file serving for /static/.
14. Fixed S3 endpoint handling for blank AWS_S3_ENDPOINT_URL.
15. Enabled signed S3 URLs for private media by using AWS_QUERYSTRING_AUTH=true.

5. Major Problems Found During Deployment And Fixes

Docker install issue:

- Problem: Docker apt repo command was pasted over multiple lines and malformed /etc/apt/sources.list.d/docker.list.
- Fix: Recreated docker.list as one clean line with Ubuntu jammy and Docker GPG key path.

Django 400 on local curl:

- Problem: curl http://127.0.0.1:8000 returned 400 because ALLOWED_HOSTS did not include 127.0.0.1.
- Fix: Tested with Host: api.toxicreels.com header. This confirmed Gunicorn worked.

Admin CSS missing:

- Problem: Django admin loaded without CSS under DEBUG=false.
- Cause: Nginx was not serving collected static files, and compose used a Docker named volume not readable by host Nginx.
- Fix: Changed compose to bind mount ./backend/staticfiles:/app/staticfiles and added Nginx location /static/ alias.

S3 upload 500: Invalid endpoint:

- Problem: blank AWS_S3_ENDPOINT_URL was passed to boto3 as an empty string.
- Fix: settings.py now converts blank endpoint to None.

Images not displaying:

- Problem: S3 bucket is private while AWS_QUERYSTRING_AUTH=false produced unsigned URLs.
- Fix: Set AWS_QUERYSTRING_AUTH=true so API returns signed S3 URLs.

Google OAuth invalid_client:

- Problem: Google reported OAuth client not found.
- Likely cause: Vercel VITE_GOOGLE_CLIENT_ID does not match an active Google OAuth web client, or the OAuth client was deleted/recreated.
- Fix path: verify Vercel production env and Google Cloud Authorized JavaScript origins.

6. Current Branch Strategy Recommended

main:

- Represents what is deployed or deployable to Lightsail production.
- Should only receive tested and reviewed changes.

staging:

- Mirrors main initially.
- Use for production-like tests before merging or promoting to main.
- Can point to the same production-like environment variables or later a staging backend.

develop:

- Mirrors main initially but becomes the active integration branch for local work.
- Use local .env, local Postgres/MinIO compose, and feature branches off develop.
- Merge develop into staging when ready for production-like validation.
- Merge staging into main when ready to deploy.

7. Code Observations And Risks

1. Secret rotation is still required.

Real secrets were pasted during the setup process. Rotate AWS access keys, Google OAuth secret, Resend API key, Django secret key, and database password before long-term production use.

2. Google OAuth is a configuration issue, not a Django endpoint issue.

The frontend Google button uses VITE_GOOGLE_CLIENT_ID. The backend verifies the ID token audience against GOOGLE_OAUTH_CLIENT_ID. Both must be the same active Google web client ID.

3. Media delivery works but is MVP-grade.

Signed S3 URLs work for private buckets, but CloudFront is the better long-term delivery layer. Signed URLs can expire and can be awkward for caching.

4. TMDB sync and ImageField may be logically mismatched.

Admin TMDB sync assigns remote image URLs to ImageField fields. ImageField expects stored file paths or uploaded files, so this can create broken image names/paths. If TMDB sync is used, implement image download-to-storage or change fields back to URLField for remote posters.

5. Watchlist can reference unpublished content if the user knows object IDs.

Watchlist validation checks that the object exists, not that it is published. This can expose metadata for draft/pending content through watchlist detail.

6. Worker permissions need one more review.

Workers are restricted on save, but admin actions and queryset visibility should be checked so workers cannot move already-published own content back to draft or modify statuses outside intended draft/pending flow.

7. Slug collision handling is weak.

Models auto-slugify title but do not resolve duplicate slugs. Creating two movies or series with the same title can trigger a uniqueness failure.

8. No explicit health endpoint.

Production deployment would benefit from /api/health/ for Nginx, uptime checks, and deployment verification.

9. Docker production startup is manual.

Migrations and collectstatic are currently manual commands. This is acceptable for controlled MVP deployment, but deployment notes must keep that explicit.

10. localStorage JWT is acceptable for MVP but has XSS risk.

Longer term, consider HTTP-only secure cookies or stronger frontend XSS controls.

8. Immediate Next Steps

1. Rotate exposed secrets before final public launch.
2. Fix Google OAuth client configuration in Vercel and Google Cloud.
3. Redeploy Vercel after environment changes.
4. Pull latest main on Lightsail and run migrations after password reset changes.
5. Add CloudFront or make an explicit decision to stay with signed S3 URLs for MVP.
6. Fix TMDB poster import behavior if TMDB sync will be used in production.

7. Add a health endpoint.
8. Add stricter watchlist validation for published-only content.

9. Lightsail Update Commands For Current Main

Run on Lightsail:

```
cd /srv/toxicreels/Tox
git pull origin main
docker compose --env-file backend/.env -f docker-compose.prod.yml up -d --build
docker compose --env-file backend/.env -f docker-compose.prod.yml exec web python
manage.py migrate
docker compose --env-file backend/.env -f docker-compose.prod.yml exec web python
manage.py collectstatic --noinput
sudo nginx -t
sudo systemctl reload nginx
```

10. Vercel Production Environment

Set in Vercel project settings:

```
VITE_API_BASE_URL=https://api.toxicreels.com/api
VITE_MEDIA_BASE_URL=https://api.toxicreels.com
VITE_GOOGLE_CLIENT_ID=<active Google web client ID>
```

After changing Vercel env, redeploy without build cache.

11. Google OAuth Production Checklist

In Google Cloud Console:

- OAuth consent screen must be configured and published for external users.
- Authorized domain should include toxicreels.com.
- OAuth Web Client Authorized JavaScript origins must include:
 - https://toxicreels.com
 - https://www.toxicreels.com
- Vercel VITE_GOOGLE_CLIENT_ID must equal backend GOOGLE_OAUTH_CLIENT_ID.
- If invalid_client remains, create a new Web application client and update both Vercel and Lightsail env.

12. Final State Assessment

The project is now production-connected at an MVP level. The backend is live, admin works, S3 uploads work, private S3 images work after signed URLs, and the frontend can connect once Vercel env and Google OAuth are corrected. The codebase is structurally sound enough to continue from develop with staging/main promotion, but the security and media delivery hardening items should not be ignored.